3강

PoCaT 2기 · 바이브 코딩

AI를 더 잘 부리는 법(스킬)과 남이 볼 수 있는 주소(배포)

오늘 끝나면 손에 남는 것

① 직접 깔아서 써 본 **스킬**

② 남에게 보낼 수 있는 **주소**

시작 전에 — 폰을 꺼내 주세요



3223570

Scan the QR code
to join anonymously

1 QR을 찍는다
또는 qr-chat.physical-spark.com/r/3223570

2 가입도 로그인도 없다
이름이 안 남는다 — 편하게 눌러도 된다

왜 하나 답을 보고 오늘 진행을 정한다
배포를 해본 사람이 몇인지에 따라 설명 깊이가 달라진다

01 오늘 다루는 것은 2가지입니다

AI를 다루는 쪽과, 만든 것을 세상에 내놓는 쪽

1 **스킬**
— AI에게 규격서를 붙인다
한 번 설치하면 계속 적용된다.
도구를 바꿔도 따라온다

2 **배포 — 인터넷에 올린다**
노트북을 꺼도 주소가 살아 있다

3 **그다음은 각자 제작**
만드는 동안 강사 2명이 순회한다

4 **마무리**
되는 사람은 주소까지 확보하고 간다

지난주 결과물은 아직 내 노트북 안에 있습니다

만든 것과 남에게 보여줄 수 있는 것은 다르다

> **용어** · 목업 데이터(mock data) = 진짜 데이터가 붙기 전, 모양만 맞춰 넣어둔 임시 데이터

지난주	한계	오늘
화면 뼈대 + 목업 데이터 20건	로컬에서만 열린다 노트북이 곧 서버였다	공개 주소로 꺼낸다 주소가 있어야 검증을 받는다

스킬 = AI에게 붙이는 작업 규격서

「이런 일은 이렇게 처리하라」를 문서로 적어 둔 폴더 하나

> **용어** · 폴더 안에 **SKILL.md** 파일이 있으면 그것이 스킬이다. 확장자 md = 마크다운, 서식이 붙는 일반 텍스트

1 **한 번 설치하면 지시가 남는다**
매번 같은 조건을 다시 입력하지 않아도 된다

2 **필요할 때만 불려 나온다**
배포를 물으면 배포 규격서만 열린다 — 항상 켜져 있는 것이 아니다

3 **여러 개를 함께 둬도 된다**
서로 간섭하지 않는다

매번 입력하는 것과 무엇이 다른가

프롬프트는 그때의 말이고, 스킬은 남아 있는 기준이다

	매번 프롬프트	스킬
적용 시점	지시할 때마다 다시 쓴다	설치 이후 계속
누락	길어질수록 조건을 빠뜨린다	입력하지 않아도 유지된다
일관성	어제와 오늘의 결과가 다르다	같은 기준으로 반복된다
이식	도구를 옮기면 다시 복사	폴더째 따라온다

어떤 도구를 쓰든 그대로 동작합니다

스킬은 특정 회사의 기능이 아니라 공개된 규격이다

> **왜 중요한가** · AI 도구는 해마다 바뀐다. 규격을 익히면 **도구가 바뀌어도 쌓은 것이 남는다**

클로드 코드	코덱스	제미나이	커서	코파일럿
Claude Code	Codex	Gemini CLI	Cursor	Copilot

규격 — **작년 말 공개 형식으로 정해졌다**
만든 곳이 달라도 폴더 구조가 같다

그래서 — **한 번 받으면** 어디서든 쓴다
도구를 갈아타도 설정이 따라온다

어디서 받는가

남이 만든 규격서를 내 AI에 붙이는 일이라 출처를 본다

> **용어** · 별(⭐) = 깃허브에서 그 저장소를 북마크한 사람 수. 많다는 것은 그만큼 검증을 거쳤다는 뜻

깃허브 — **제작자가 공개한 저장소**
가장 흔한 배포처다

등록소 — **스킬만 모아 둔 사이트**
보안 점검 결과가 함께 표시된다

판단 기준 — **별 수 · 마지막 수정 날짜 · 제작자**
반년 넘게 손대지 않은 스킬은 지금 동작하지 않을 수 있다

설치는 한 줄입니다

터미널에 붙여넣기만 하면 끝난다

> **용어 · npx** = 내려받아 곧바로 실행하는 명령. 따로 설치할 것이 없어 한 줄로 끝난다

```
npx skills add NomaDamas/k-skill
```

1
터미널에 붙여넣기
클로드 코드를 쓰는 그 화면

2
받을 항목 선택
전체 또는 필요한 것만

3
설치 완료
이후 자동으로 불려 나온다

k-skill — 한국 서비스에 붙는 123개

국내 서비스와 공공데이터를 AI가 곧바로 호출하도록 미리 연결해 둔 모음

왜 필요한가 · 공공데이터는 보통 **키 발급 → 문서 해독 → 응답 파싱**을 거쳐야 쓴다. 스킬은 그 절차를 안에 넣어 둔 것이다

부동산
실거래가 · 공시가격 · 등기

생활
날씨 · 급식 · 도서관 · 공휴일

일·돈
채용공고 · 장학금 · 정부지원 · 주식

취미
KBO 야구 · K리그 · 영화 · 공연

고르기 어려우면 이 5개부터

스킬	가져오는 것	이런 주제에
korean-scholarship-search	장학 공고 · 한국장학재단	**장학금·공모전**
job-posting-match	채용 공고 + 적합도	**취업 대시보드**
daangn-jobs-search	당근알바 공고	**알바 찾기**
naver-news-search	뉴스 제목·링크·요약	**뉴스 모아보기**
korean-cinema-search	상영표·잔여석	**영화·나들이**

10 더 찾으려면 카탈로그에서

분야별로 정리된 공개 목록이 1000여개나 있다

awesome-agent-skills **공개 목록 저장소** — 스킬 1,000여 개가 분야별로 정리돼 있다

검색어 **영어로 검색한다** — 「뉴스」보다 news, 「지도」보다 map.
목록 자체가 영어로 쓰여 있어 한글로는 결과가 거의 안 나온다

선택 기준 별 수와 마지막 수정 날짜. 앞 장과 같은 기준이다

대부분은 무관 **개발자·기업용이 대부분** — 엔비디아·마이크로소프트 것만 288개다.
소수만 골라 쓴다 — 다음 장의 5개가 그것이다

주제를 안 가리는 5개

스킬	하는 일	언제
frontend-design	배치·색·글자를 다듬는다	**다음 주 — 디자인**
webapp-testing	대신 눌러 보며 점검	**배포 직전**
skill-creator	내 규칙을 스킬로 만들어 준다	**쓰다 익숙해지면**
pptx	발표 자료를 만들고 고친다	**마지막 주 — 발표**
xlsx	엑셀을 읽고 만든다	**데이터를 표로 다룰 때**

11 스킬이 막아 주는 것
— AI의 반복되는 4가지 실수

1 **넘겨짚는다**
지시하지 않은 것까지 만들어 낸다

2 **과하게 설계한다**
간단한 작업을 크게 벌인다

3 **범위를 벗어난다**
한 곳을 고쳐 달라니 전체를 바꾼다

4 **확인 없이 완료라 한다**
검증하지 않고 끝났다고 보고한다

오늘 각자 스킬 1개 이상

설치까지가 아니라 한 번 써 보는 것까지가 과제다

1
내 주제에 맞는 것 탐색
k-skill 또는 카탈로그

2
설치
명령 한 줄 또는 AI에게 지시

3
실제로 지시해 본다
전과 무엇이 달라졌는지 확인

4
노션에 한 줄
설치한 것과 사용 소감

6주 뒤 이 기록이 **「무엇을 다룰 줄 아는가」의 목록**이 된다

13 배포

= 실행 환경을 내 컴퓨터 밖으로 옮기는 일

파일을 공유하는 것이 아니라, 돌아가는 화면을 올려 두는 것이다

용어

- 배포(deploy) = 만든 것을 실제로 접속 가능한 자리에 올려 두는 일
- 서버 = 24시간 켜져 있는 남의 컴퓨터

지금 · 로컬

내 컴퓨터에서만 열린다

노트북을 닫으면 접속이 끊긴다.
보여 주려면 옆에 앉아야 한다

배포 후 · 서버

주소 하나가 생긴다

누구나 링크로 들어온다.
전원을 꺼도 살아 있다

왜 완성 전에 먼저 올리는가

공개된 주소가 있어야 피드백이 들어온다

1

보여 줄 수 있어야 고칠 곳이 보인다

혼자 보면 전부 괜찮아 보인다

2

포트폴리오에 들어가는 것은 링크 하나

평가자는 코드를 열지 않는다. 주소를 눌러 30초 안에 판단한다

3

미완성 상태로 올려도 된다

완성한 뒤 올리는 것이 아니라, 올려 두고 고쳐 나간다

먼저 — 내 코드의 위치를 확인합니다

배포에서 막히는 지점의 절반이 여기다

프로젝트 폴더 — **어디에 만들었는가** — 바탕화면인지 다운로드인지 확인한다.
AI가 만들어 줬어도 위치는 본인이 알아야 한다

index.html — **진입 파일** — 이 파일이 폴더 안에 있어야 한다

모르겠으면 — 터미널에서 「지금 이 폴더의 경로를 알려 줘」라고 지시한다.
나온 경로를 적어 둔다 — 오늘 계속 쓴다

16 올리는 경로는 3가지, 결과는 같습니다

어느 쪽으로 가도 접속 가능한 주소 하나가 나온다

경로	방식	오늘
AI에게 지시	「이 폴더를 vercel로 배포해 줘」라고 문장으로 요청	**이 방법으로 진행**
터미널 명령	폴더에서 명령 한 줄 실행	동일한 결과
끌어다 놓기	브라우저 창에 폴더를 끌어 놓는다	막혔을 때

AI에게 시키는 이유 — **30명의 폴더 구조가 전부 달라서** 앞에서 시연해도 그대로 따라오기 어렵다

이 문장을 그대로 사용하세요

막힐 지점을 미리 지시문에 넣어 두는 것이 핵심이다

"이 폴더를 **Vercel**에 배포해 줘.
계정이 없으면 만드는 방법부터 알려 주고,
끝나면 **주소를 알려 줘**."

앞 조건

계정 유무를 미리 짚는다

없으면 AI가 계정이 있다고 가정하고 진행하다 중간에 멈춘다

뒤 조건

결과물을 명시한다

「주소를 알려 줘」가 없으면 배포만 하고 알려 주지 않는다

18 첫번째 방법이 잘 안되면 — https://vercel.com/drop (드래그 앤 드롭)

10분 해 보고 안 되면 이 경로로 넘어간다

1
vercel.com 가입
이메일만 있으면 된다

2
폴더를 창에 끌어 놓는다
파일 탐색기에서 그대로

3
주소가 생성된다
30초

⚠ 단점 — 올릴 때마다 **주소가 새로 생긴다.**
다음 주에 고쳐 올리면 지난번에 보낸 링크는 옛 화면에 멈춰 있다

주소가 2개 나오는 이유

배포할 때마다 그 시점의 화면이 따로 보존된다

고정 주소 · production

항상 최신 배포를 가리킨다

고쳐서 다시 올려도 주소는 그대로다.
포트폴리오·이력서에 적을 주소이고,
남에게 보낼 것도 이쪽이다

임시 주소 · preview

그 배포 한 건만 가리킨다

올릴 때마다 새로 생긴다.
고치기 전 화면을 남겨 두거나,
바뀐 점을 견줄 때 쓴다

헷갈리면 **짧은 쪽이 고정 주소**다

21 보통은 여기서 자주 막힙니다

index.html

진입 파일 — 서버는 이 이름을 먼저 찾는다.
이름이 다르면 보여줄 것이 없다고 판단한다

루트 폴더

한 겹 더 들어감 — 폴더 안에 폴더가 또 있으면 진입 파일을 못 찾는다

Index ≠ index

대소문자 — 서버는 리눅스라 둘을 다른 파일로 본다.
맥·윈도우는 구분하지 않아 내 컴퓨터에선 열렸던 것이다

환경 변수

키를 넣었으면 다시 배포한다 — 배포 시점에 한 번만 읽히기 때문이다

해결

에러 메시지를 통째로 복사해 AI에게 넘긴다.
읽고 해석하려 하지 말고 그대로 붙여넣는다

주소가 나오면 2가지

기록해 두는 것과, 남에게 보내 보는 것

1 **노션에 기록한다**
「결과물 주소」 칸. 기록하지 않으면 다음 주에 다시 찾는다

2 **1명에게 보낸다**
친구든 가족이든 상관없다. 남이 보면 안 보이던 것이 보인다

3 **반응을 한 줄 남긴다**
「무엇을 하는 건지 모르겠다」도 좋은 기록이다 — 화면이 스스로 설명하지 못한다는 뜻이다

오늘 남길 것은 2가지

전원이 스킬, 되는 사람은 주소까지

1 **설치해서 사용해 본 스킬**
노션에 이름과 소감 한 줄.
전원 해당

2 **배포 주소**
되는 사람까지.
안 되면 다음 주에 이어서 한다

3 **막힌 지점 기록**
「폴더를 못 찾았다」 한 줄이면 된다

4 **왜 남기나**
기록이 없으면 다음 주 준비를 못 한다

파트 2 · 실습

순서는 스킬 → 배포

막히면 5분 안에 손을 든다 · 강사 2명이 순회한다

1 **내 주제에 맞는 스킬을 찾아 설치**
k-skill 또는 카탈로그에서

2 **한 번 사용해 보고 노션에 한 줄**
여기까지는 전원

3 **되는 사람은 배포까지**
「이 폴더를 배포해 줘」